WPT Day 1

Shivam Zanjurne · Wed 21 – Thu 22 Oct 26 · Servers, client–server architecture, frontend vs backend, HTTP, HTML tables & forms, and first CSS — explained from first principles

---

Table of Contents

1. What a Server Is and Why Servers Are Necessary
2. Client-Server Architecture
3. Frontend vs Backend Development
4. HTML, CSS and JavaScript: The Three Frontend Fundamentals
5. Libraries and Frameworks
6. Backend Technologies and Database Choices
7. HTTP Requests: URL and Method
8. HTTP Responses: Status Codes
9. HTML Tables: Structure, Borders, Padding and Spacing
10. HTML Forms: Purpose, action and method
11. Input Types, Labels and Line Breaks
12. Radio Buttons, Dropdowns, Checkboxes and Textarea
13. Buttons: Plain, Submit and Reset
14. CSS Introduction: Selectors, Properties and Where Styles Live
15. Combined Worked Example: The Aligned Form
16. Quick Revision Checklist
17. Practice Exercises
18. Practice Answers

What a Server Is and Why Servers Are Necessary

Start from the actual problem

You build an application on your laptop. It works perfectly — on your laptop. Now ask the only question that matters for a web application: **how does a stranger in another city, on another network, at 3 AM while your laptop is shut, open your application and use it?**

Almost every idea in this chapter exists to answer that one question. A server is the answer the industry settled on, and once you see *why* the naive alternatives fail, the definition becomes obvious rather than something to memorise.

Try the naive alternatives first:

- **Email the code to everyone / put it on a pen drive.** Now there are thousands of copies. Whose copy holds the real data? When one user places an order, how does another user ever see it? There is no single source of truth.

- **Let people connect directly to your laptop.** Your laptop sleeps, runs out of battery, loses Wi-Fi, gets a new IP address when the router restarts, and sits behind your home router which by default refuses incoming connections from the internet. Even if none of that happened, one laptop cannot serve ten thousand people at once.
- **Store the data separately with each user.** Then there is no shared reality at all — no common product catalogue, no common order list, no login that works from a second device.

So a real web application needs a machine that is (a) always on, (b) reachable by anyone at a stable public address, (c) powerful enough for everyone's combined workload, and (d) the *single* home of both the program and its data. That machine is a server.

Definition

A **server** is a special-purpose computer connected to the internet that **hosts applications anyone in the world can access**.

> **Exam:** A server is not a different *kind* of computer. Architecturally it is identical to a local machine — it has a CPU, RAM, a hard disk / SSD, and a processor. The difference is **capacity and purpose**, not architecture.

Unpack each word of the definition, because each word is doing work:

- **Computer** — ordinary computing hardware: CPU to execute code, RAM to hold what is being worked on right now, disk to hold things permanently. Nothing magical happens "in the cloud"; the cloud is other people's servers.
- **Connected to the internet** — reachable from outside its own building. A powerful machine in a locked room with no network cable serves nobody.
- **Hosts applications** — it *runs* the application's code and *stores* the application's data, continuously, whether or not any particular user is currently looking.
- **Anyone in the world can access** — access is not limited to the developer's desk, office, or city. If the address is known and the application permits it, a request from anywhere is served.

Server vs local machine

Aspect	Local machine (laptop/desktop)	Server
Basic architecture	CPU, RAM, storage	CPU, RAM, storage — the same
Processing power	Consumer-grade, sized for one user	Far higher; sized for thousands of simultaneous users

Aspect	Local machine (laptop/desktop)	Server
RAM and storage	Limited (e.g. hundreds of GB)	Very large; can reach hundreds of thousands of TB across a fleet
Uptime expectation	Sleeps, shuts down, restarts for updates	Runs 24/7 without interruption
Power and network	Home/office supply; outages are normal	Engineered with backups; outages are treated as emergencies
Intended workload	One person's foreground tasks	Continuous, high-volume requests from strangers

The key insight is that the boundary is not a law of physics. The lecture makes this point deliberately:

A local laptop **can** technically be turned into a server, but only if all of the following hold at the same time:

1. It runs **24/7 without interruption** — no sleep, no shutdown, no lid closing.
2. There is **no internet or electricity outage** — ever, because every outage is downtime for every user.
3. It has **enough hardware** to accommodate the entire application *and* its data *and* all simultaneous users.

Fail any one condition and the "server" stops being a server in practice. This is why hobby experiments do run servers on laptops and old PCs, and why no business you have heard of does.

The two reasons a server is necessary

Once an application is built, a server is required for **two distinct reasons**. Keep them separate — examiners like to see whether you merge them into one vague "because scale".

Reason 1: Accessibility

- Applications are **developed** on a local machine. Development is a private activity; using the application is a public one.
- It is not practically possible to give the entire world access to a developer's local machine — it is behind a router, on a changing address, and off half the time.
- A server provides a **publicly accessible, always-on location** to host the application. One stable address, reachable by anyone, all the time.

Background (first principles): every machine on the internet is identified by an **IP address** (covered properly in the HTTP chapter). Your laptop usually has a private, temporary address handed out by your router; a server is given a stable public address precisely so that strangers' requests can find it reliably, day after day.

Reason 2: Scale of data and processing

- Large applications like Amazon, Facebook, and Flipkart hold **hundreds of thousands of TBs of data**.
- A local machine simply cannot store that volume, and even the fraction it could store it could not *process* fast enough when lakhs of users ask questions at the same moment.
- Servers are purpose-built for **massive data storage and high-throughput processing** — many machines working together, each far stronger than a consumer laptop.

Put together: accessibility is about *reach* (can the world get to it?), scale is about *weight* (can it carry the world's combined load?). Both limitations make a dedicated server not just convenient but **essential**.

What lives on the server

A server holds **two things simultaneously**:

1. **The application itself** — all the code and logic: what happens on login, how an order is created, how a comment is posted.
2. **The database** — all the application's data: users, products, orders, carts, comments, everything the application must remember between visits.

And consequently, **all business operations execute on the server**: login, logout, creating orders, cancelling orders, posting comments, searching products — everything. Your browser asks; the server decides and does.

Why must code and data live together on the server rather than, say, data on each user's phone? Return to first principles: an order placed on one device must be visible on another; two users must never be sold the same last item in stock from two stale local copies. One central machine holding one authoritative copy of the data is what makes a shared, consistent reality possible. (Later modules will show the database often runs on its *own* server machine — that is an engineering refinement. Architecturally it is still "on the server side", never on the user's device.)

Facebook, Amazon, and Flipkart all follow this same model regardless of scale. The difference between your course project and Flipkart is capacity and polish, not architecture.

> **Exam:** "Why is a server necessary?" expects **both** reasons — accessibility and scale of data/processing. Writing only "to store a lot of data" loses the accessibility half.

> **Pitfall:** Do not define a server as "a powerful computer" and stop. Power is the least interesting part. The defining properties are: always on, publicly reachable, hosts both the application and its database, and executes all business operations.

Client-Server Architecture

The fundamental model

The **client–server architecture** is the fundamental model of *every* web application, regardless of technology stack. React or Angular, Java or JavaScript, MySQL or MongoDB — underneath, all of them are this one pattern. Learn it once, here, and every later topic in WPT is a detail of this diagram.

First principles: who are the actors?

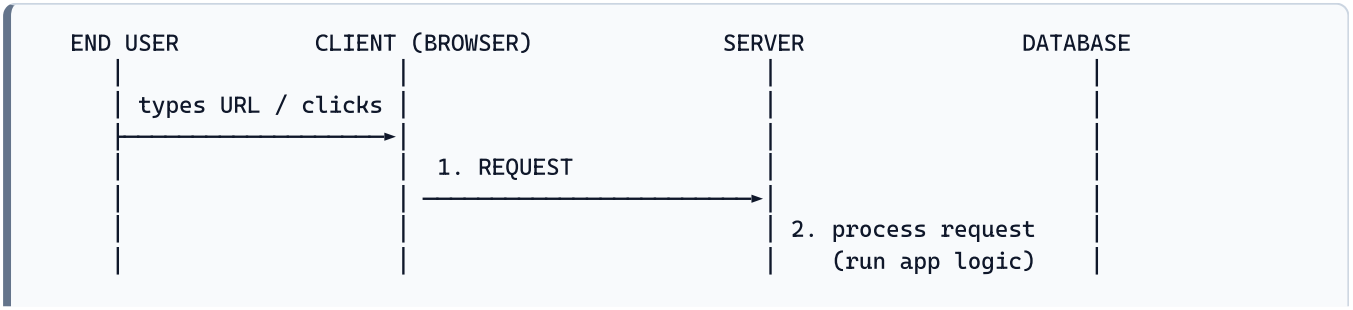
Three actors, and the lecture insists on distinguishing them precisely:

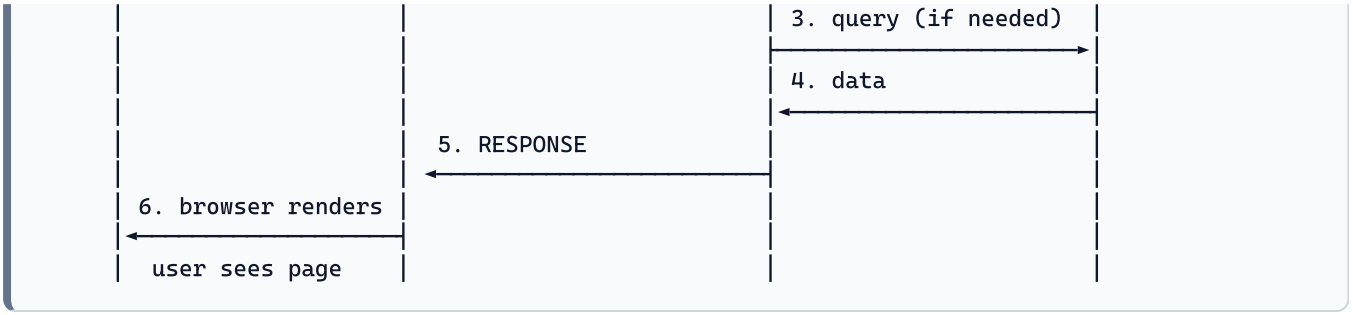
- **End user** — the human. Types, clicks, reads.
- **Client** — the **web browser** (Chrome, Edge, Firefox…). The browser is a program whose job is to make requests on the human's behalf and display what comes back.
- **Server** — the always-on machine hosting the application and database.

> **Exam:** "The web browser is the client, not the end user." The end user interacts with the browser; the browser interacts with the server. In the lecture's words: "*We are end users, not clients.*" This distinction is asked directly.

Why does the distinction matter beyond pedantry? Because everything technical — requests, methods, status codes, rendering — happens between the browser and the server, two *programs* speaking a protocol. The human never speaks HTTP; the browser translates human actions (typing a URL, clicking Search) into HTTP requests and translates responses back into pixels.

The flow, step by step





Walk the lecture's own example through it — searching "iPhone 16" on an e-commerce site:

1. The end user types a URL (e.g. `flipkart.com`, `youtube.com`) into the browser and presses Enter, or types "iPhone 16" into the search box and clicks Search.
2. The **browser (client) sends a request** to the server at that address: in effect, "show me results for iPhone 16".
3. The **server accepts the request and processes it** — the application logic on the server runs.
4. Data is needed, so the **server queries the database** for products matching "iPhone 16".
5. The **database returns the relevant data** to the server application.
6. The **server sends a response back to the client** — the results, packaged for display.
7. The **browser displays the response** to the end user, who sees a page of phones.

Notice what the database never does: it never talks to the browser. The browser never talks to the database. All communication is browser ↔ server, and server ↔ database. This layering is why the client-side code can never be trusted with business rules — a theme that returns in form validation later: the browser suggests and displays, the server decides.

The lecture demonstrated this live with `fast.com`: typing the address sent a request to the fast.com server, which responded with the full page the class then saw. Every URL you have ever typed was this same seven-step dance.

A server without a client has no purpose

Without a client, the server **cannot act at all** — it does nothing until a request arrives. The server is fundamentally *reactive*: it waits, listens, and responds.

The lecture's analogy: **a restaurant with zero customers cannot prepare any food, no matter how large its kitchen.** The kitchen's capacity is irrelevant until an order arrives; likewise a server's power sits idle until a request gives it something to do. This is not a limitation — it is the design. One server must serve lakhs of unrelated clients whose requests arrive unpredictably, so it waits and reacts rather than acting on its own schedule.

Two corollaries worth internalising:

- **An internet connection is mandatory** for any client–server communication. No connection, no request, no response — the architecture simply does not exist without the network between the two programs.
- **This architecture is universal.** It applies to every web application, from a two-page course project to enterprise systems, and it is **not tied to any specific technology**. Technologies change every few years; this diagram has not changed in decades.

Background (first principles): notice the request–response rhythm — the client always speaks first, the server only replies, and after replying the server typically forgets the exchange (HTTP is *stateless*; remembering "you are logged in" is extra machinery built on top, which later sessions will meet). Even without the vocabulary, you can already see it in the diagram: every arrow pair begins with the client.

> **Pitfall:** Drawing the end user as the client, or drawing an arrow from the browser straight to the database. Both are wrong in exactly the ways this chapter warned about: the browser is the client, and the database only ever answers the server.

Frontend vs Backend Development

Two aspects of one application

Building any complete web application requires developing **two distinct aspects**. The split is exactly the client–server split from the previous chapter, seen from the developer's chair instead of the network's.

Frontend: what the user sees

The **frontend** is everything the user sees and interacts with **in the browser**:

- Navigation bars and footers
- Buttons, links, sliders
- Input fields and forms
- Images and other media

Essentially, the entire **UI (user interface)** of the application. If a user can point at it on the screen, it is frontend. Frontend code runs on the *client* — it is delivered to the browser and executed/displayed there.

Backend: what happens behind the scenes

The **backend** is all the processing and data management that happens **behind the scenes**, on the server. It consists of **two sub-parts**, and naming both is important:

1. **Application logic** — the code that accepts requests, runs business rules, and generates responses. ("Is this password correct? Does this user have permission? What is the total of this cart after the coupon?")
2. **Database** — the storage layer that holds all of the application's data, permanently, across visits and devices.

Notice the backend mirrors what the server chapter said lives on the server: application + database. That is not a coincidence; "backend" is the name for the server-side half of the work.

Both are equally important

The lecture addresses the industry debate directly: arguments about which side is harder or more important are **misleading**. Difficulty depends on the **complexity of the specific feature being built, not the layer**. A deceptively simple-looking search box can hide brutal backend work (ranking lakhs of products in milliseconds); a visually simple checkout page can hide brutal frontend work (flawless behaviour across devices, browsers, and half-filled states). Neither half is the "real" programming.

The complete formula:

```
COMPLETE WEB APPLICATION  =  UI  +  APPLICATION LOGIC  +  DATABASE
                             └─ frontend ─┘  └──────── backend ────────┘

Remove any one term and the application stops being an application:
  • No UI              → nobody can use it
  • No application     → nothing decides or computes anything
    logic
  • No database        → nothing is remembered between visits
```

> **Exam:** "Backend consists of which two sub-parts?" — **application logic and database**. And the full application is **UI + application logic + database**; all three are required.

HTML, CSS and JavaScript: The Three Frontend Fundamentals

There are only three — and there are no alternatives

Three core technologies underpin **all** frontend development. Every frontend tool you will ever meet — including the libraries in the next chapter — is built on top of these three, because at the end of

the chain the browser only understands these three. They are not three competing options; they are three cooperating layers, each answering a different question about a page:

- **HTML** answers: *what is on the page?* (structure and content)
- **CSS** answers: *what does it look like?* (presentation)
- **JavaScript** answers: *what does it do?* (behaviour)

HTML — HyperText Markup Language

HTML **creates the structure and skeleton of a web page**. Everything visible in the browser is rendered through HTML: images, videos, audio controls, buttons, input fields, dropdowns, checkboxes, radio buttons, tables, links, navigation bars, footers.

The cleanest way to feel its role: **without HTML, nothing appears on the page at all.** There is no page. HTML is the skeleton — the bones that say "there is a heading here, a paragraph there, a button below that".

But a skeleton is all it is. **HTML alone produces a plain, unstyled black-and-white page** — every element present, in order, with the browser's bare default appearance. Correct, complete, and ugly.

Background (first principles): HTML is a **markup** language, not a programming language — it *describes* structure ("this is a heading", "this is a cell") rather than computing anything. Keep that sentence ready; it is a favourite interview opener, and Day 1's tables and forms are exactly this description work in action.

CSS — Cascading Style Sheets

CSS is responsible for **all visual styling**: background colours, font colours, font styles, button colours, layout, spacing. It takes HTML's skeleton and decides how every bone is presented.

Without CSS, a page is a plain, unformatted document with no visual appeal. The lecture's phrase is worth keeping verbatim because it is exactly right: CSS is **"a beauty parlour for your web page."** Same page underneath; transformed appearance on top. Styling is not vanity, either — attractive, well-spaced pages encourage users to spend more time on them, which is a business outcome, not decoration.

The mechanics of CSS (selectors, properties, where styles are written) begin on Day 2 of the lecture and get their own chapter later in these notes.

JavaScript

JavaScript **adds logic and interactivity on the client side, executed directly in the browser**. The three-layer contrast in one line:

- HTML can **display** a button.

- CSS can make that button **look attractive**.
- Only JavaScript makes **something happen when the user clicks it**.

Common uses named in the lecture:

- **Form validation** — required fields filled? Email in the correct format? Password obeying the rules? Checked *in the browser, instantly*, before anything is sent.
- **Dynamic content updates** — changing what is on the page without a full reload.
- **Handling user events** — clicks, typing, selections: noticing them and reacting.

Think about why client-side validation exists at all, from first principles: the alternative is sending every half-filled form across the network to the server and waiting for "you forgot your name" to come back. JavaScript in the browser catches it in zero network time. (It never *replaces* server-side checks — the browser's code can be bypassed, which is why the server re-validates. Client-side is for speed and user experience; server-side is for truth.)

Java vs JavaScript — completely unrelated

> **Exam: Java and JavaScript are completely unrelated technologies** despite the naming similarity. In the lecture's words: "*Comparing Java to JavaScript is like comparing car to carpet.*"

They share superficial syntax in some areas (braces, `if`, `for`), but their purposes, ecosystems, and histories are entirely different. One is a general-purpose language heavily used on servers (and in your earlier modules); the other was born to make web pages interactive in the browser. The naming history — why the misleading name exists at all — will be covered in depth when JavaScript is introduced formally. For now, file the names as a historical accident and never let one stand in for the other on an exam or in an interview.

> **Pitfall:** Writing "JavaScript is a lightweight version of Java", "JavaScript is Java for the web", or listing Java experience as JavaScript experience. All wrong, all common, all exactly what the car/carpet line is warning against.

Libraries and Frameworks

The problem with starting from zero

Writing an entire application from scratch using **only** HTML, CSS, and JavaScript is possible — the three fundamentals are sufficient in principle. It is also **extremely time-consuming and complex** in practice, because you would personally have to solve, from zero, problems every project shares:

code reusability (the same button, header, and form logic rebuilt on every page), security, and scalability as the application grows.

The lecture's analogy is a kitchen one: making gulab jamun **from raw ingredients** versus using a **premixed packet**. The premix does not change what gulab jamun is; it saves time, reduces effort, and provides proven things **out of the box**. Libraries and frameworks are the premix.

What a library or framework actually is

Libraries and frameworks are **built on top of the three core technologies, not replacements for them**. React still produces HTML, is styled by CSS, and is JavaScript throughout. What the library adds is pre-built, battle-tested solutions to the recurring problems — component structure, reusability, state handling, routing — so your effort goes into *your* application's unique parts.

This has a hard consequence the lecture states as a principle: **libraries make development life easier, but understanding HTML, CSS, and JS fundamentals is non-negotiable.** When the premix misbehaves — and it will — only someone who knows the ingredients can tell why. This is precisely why the course spends Day 1 on raw HTML tables and forms instead of opening React on day one.

The common frontend libraries and frameworks

Tool	What it is	What it gives you
React.js	JavaScript library for UIs	Component structure, reusability, and scalability without building from scratch
Angular	Full JavaScript framework	A complete, opinionated toolkit for large-scale frontend applications
Vue.js	JavaScript UI framework	Another approach to component-based UIs
jQuery	JavaScript library	Simplifies DOM manipulation and event handling
Tailwind CSS	Utility-first CSS library	Pre-built styling utilities so you don't write raw CSS from scratch
Bootstrap	CSS framework	Pre-built, ready-made UI components

Read the table by layer: React/Angular/Vue/jQuery live in the JavaScript layer; Tailwind and Bootstrap live in the CSS layer. **There is no library in the HTML layer's place** — everything still bottoms out in HTML in the browser, which circles back to why the fundamentals are non-negotiable.

Course path: this course will cover **React.js and Tailwind CSS** as part of the full journey — one from the JavaScript layer, one from the CSS layer, sitting on the fundamentals you are learning now.

> **Exam:** Be able to state the core principle in one breath: libraries/frameworks are built *on top of* HTML, CSS, and JavaScript to save time and provide reusability/security/scalability out of the box — they never replace the fundamentals.

Backend Technologies and Database Choices

The server side is language-flexible

Server-side application logic can be written in **many languages**: Java, Python, .NET, PHP, C, C++, and others. The backend is defined by its *job* (accept requests, apply business rules, answer), not by a language. Companies pick based on team skills, ecosystem, performance needs, and existing systems.

JavaScript crosses over to the server

The interesting development for this course: **JavaScript can now run server-side as well — which was not the case when it was first introduced.**

- **Originally**, JavaScript was **exclusively a client-side language** — it lived in the browser and nowhere else.
- **Later enhancements** gave it the capability to execute on the server.
- The consequence: a developer can use **one language (JavaScript) for both frontend and backend**.

That is the course's choice: **JavaScript for both layers**, so no separate backend language needs to be learned. The practical payoff is real — one syntax, one mental model, and data structures that don't have to be re-learned in a second language when you cross the client–server boundary. (The runtime that makes server-side JavaScript possible gets introduced formally later in the course; Day 1 only needs the fact and its consequence.)

Databases are equally flexible

Kind	Examples	Shape of the data
Relational	MySQL, PostgreSQL, Oracle	Tables, rows, columns, strict relationships (the DBT module's world)

Kind	Examples	Shape of the data
Non-relational	MongoDB, Cassandra	Documents / flexible structures, no rigid table schema

The choice depends **entirely on the application's requirements**. And here is the point students most often get wrong:

> **Exam:** There is **no forced pairing** between backend language and database. A JavaScript backend can use MySQL; a Java backend can use MongoDB. Language and database are **independent choices**.

Who decides? The software architect

Technology-stack decisions in real IT companies are made by a **software architect**. The architect evaluates the project and the customer's requirements and selects the appropriate combination of frontend framework, backend language, and database. Because the choices are independent, the valid combinations are effectively endless:

- React.js + Java + MySQL
- Angular + Java + MongoDB
- Vue.js + JavaScript + PostgreSQL
- ...and every other permutation the requirements justify.

Course coverage: databases in this course are **MySQL** (already familiar from DBT) and **MongoDB** (the non-relational representative). You will therefore see both shapes of data before the module ends.

> **Pitfall:** Assuming "JavaScript project ⇒ must use MongoDB" or "Java project ⇒ must use MySQL". Pairings you see often in tutorials are conventions and habits, not rules. The rule is: requirements decide, architect chooses.

HTTP Requests: URL and Method

Why a request needs a structure at all

Return to the client–server diagram. The browser wants something from a server — but *which* server, out of the lakhs of servers on the internet, and *what exactly* is it asking that server to do? A

request that cannot answer both questions cannot be delivered or acted on. So every request sent from a client to a server has **two mandatory, always-present components**:

1. **URL or IP address** — *where* the request is going.
2. **Request method** — *what* the request wants done.

Neither is optional, and neither is ever absent — even when the user never sees them. This chapter takes them in turn.

Component 1: URL or IP address — the address

The URL/IP **identifies which specific server the request should be sent to**. The internet contains lakhs of servers; an address is required to reach the right one, exactly as a postal address is required to reach the right house.

It can take two forms:

- An **IP address** — the machine-level numeric address, e.g. `192.168.1.2`.
- A **human-readable URL** — e.g. `www.flipkart.com`, `youtube.com`, `amazon.in`.

Background (first principles): humans are bad at memorising numbers and machines need them, so the internet keeps a directory — the **DNS** (Domain Name System) — that translates the name you type into the IP address the network actually routes to. You work with names; underneath, every request still travels to a number. This is also why a server needs the *stable* public address from Chapter 1: a directory entry is useless if the address keeps changing.

Typing a URL in the browser **is** sending a request to that server's address — there is no separate "connect" step from the user's point of view. The lecture demonstrated this live: opening `fast.com` sent a request to the fast.com server, which responded with the full page.

Component 2: Request method — the purpose

The request method **specifies the purpose of the request**: what operation the client is asking the server to perform on the addressed resource.

Three facts about methods that are easy to miss and frequently examined:

- The method is **always present with every request, even when not visible to the user.** When you type a URL and press Enter, a method was chosen for you (GET, as you will see in the forms chapter). Invisible ≠ absent.
- Methods are **defined not by any programming language** but by the creators of the internet itself. They are **universal rules** of HTTP — the same in every language, framework, and country.

- When building the backend, **developers write code that specifies the URL and method for each operation** — every feature you build will declare "at this address, for this method, do this".

The four fundamental HTTP methods

Method	Purpose	Lecture examples	Plain-English reading
GET	Retrieve / fetch data from the server	Loading a page, searching for a product	"Give me…"
POST	Submit new data to the server	Login, signup, registration, placing an order	"Here is something new…"
PUT	Update existing data already stored on the server	Editing saved details	"Replace/change what you have…"
DELETE	Remove existing data from the server	Deleting a record	"Remove…"

Notice the symmetry: the four methods are the four things one can fundamentally do to stored data — fetch it, create it, change it, remove it. If you know what an operation *does to data*, you can choose its method without memorising a list. Additional methods exist — **PATCH, OPTIONS, and others** — and will be covered later; the four above are the foundation.

> **Exam:** Every request has **two mandatory components: URL/IP address and request method**. And the four fundamental methods with their purposes — GET (retrieve), POST (submit new), PUT (update existing), DELETE (remove) — should be recallable with one example each.

Seeing requests live: DevTools Network tab

This is not theory you must take on faith — your browser shows you every request it makes:

1. Open a site (the lecture used Flipkart) and open **DevTools → Network** tab.
2. Reload the page, then search for something (the lecture searched "Samsung S25").
3. Watch the panel fill with **multiple outgoing requests** — one page involves many requests, not one.
4. Select any request: it has a visible **URL** and an associated **request method** (POST was visible in the examples shown in class).

Do this once yourself. The realisation that a single page-load is dozens of request–response pairs, each with its own URL and method, makes everything in the next chapter concrete.

> **Pitfall:** Believing the method only exists when a form or button is involved. Typing a URL, clicking a link, reloading a page — all of these send requests with methods (GET). The method is part of *every* request; forms merely let *you* choose it.

HTTP Responses: Status Codes

A reply must say more than the data

The server finishes processing a request and sends back a **response**. But processing is **not guaranteed to succeed**: code can crash, server resources can be exhausted, or the client can send a bad request. So a response that contained only data would be broken by design — on failure there is no data, and the client would be left guessing what happened.

Therefore the response always contains **two things**:

- **Data (optional)** — the actual content requested, such as a web page or product results. Present when there is something to give.
- **Status (always present)** — a signal telling the client **exactly what happened on the server**. Present in *every* response, success or failure.

The two parts of status

Status is composed of two predefined parts that always travel together:

1. **Status code** — a numeric integer (e.g. `200`, `404`, `500`).
2. **Status message** — a short human-readable description corresponding to the code (e.g. `OK`, `Not Found`, `Internal Server Error`).

The code is for programs (compact, exact, language-independent); the message is for humans reading logs and DevTools. The lecture's analogy: it works like a **WhatsApp profile status**, which communicates your current situation to others without a direct conversation — the server uses status codes to communicate its current state to all clients, in a vocabulary everyone has agreed on in advance.

That last phrase is a rule, not a flourish:

> **Exam:** Status codes and their meanings are **predefined and universal; developers cannot change what a code means.** A backend developer's job is to **send the correct status code in every scenario they handle** — choosing from the standard vocabulary, never inventing meanings.

The status code families

Individual codes are memorisable in families, because the **first digit encodes whose "fault" / what kind of outcome it was**:

Range	Family	Meaning	Who/what is responsible
200–299	Success	The request was processed successfully	Nobody — it worked
300–399	Redirection	The client is being sent elsewhere for the resource	Neither side is at fault; it's a signpost
400–499	Client-side failure	The request itself was wrong	**The client / the request**
500–599	Server-side failure	The server failed while handling a (possibly fine) request	**The server**

Now the specific codes from the lecture:

2xx — Success

Code	Name	Meaning
200	OK	Request processed successfully — the standard success reply
201, 202, ...	—	Also indicate successful outcomes (finer shades of "it worked", detailed later)

3xx — Redirection

The **300–399** range covers **redirection responses**: the resource lives somewhere else now, and the client is told where to go. (Day 1 needs only the family's meaning.)

4xx — Client-side failure (the fault lies with the request, not the server)

Code	Name	Meaning
400	Bad Request	The client sent malformed or incorrect data

Code	Name	Meaning
401	Unauthorized	The requested resource requires authentication; the client did not authenticate first
402	Payment Required	Payment is required for the resource
404	Not Found	The requested resource does not exist at the given URL — the widely recognised one

5xx — Server-side failure (the fault lies with the server)

Code	Name	Meaning
500	Internal Server Error	The server code crashed or encountered an unhandled error
501, 502, ...	—	Cover additional server-side failure scenarios

The lecture's realism check applies to the whole 5xx family: "*Trust your code, but know it will break sometimes.*" A 500 is not a hypothetical — it is the server honestly reporting "my side broke", which is infinitely more useful than silence or a fake success.

The lecture saw this live, too: DevTools on Flipkart showed some requests returning **200 OK** and others returning **500 errors in real time** — real production systems emit both, constantly, and the Network tab lets you watch.

Reading a status code like a developer

Given any code, the first digit alone tells you the story before you look the code up: *2 — fine. 3 — go elsewhere. 4 — I (the client) asked wrongly; fix the request. 5 — the server broke; fix the server.* In debugging this splits the world instantly: a 4xx sends you to inspect what you sent; a 5xx sends you to the server logs. Sending a 200 when something failed, or a 500 when the client sent garbage, actively misleads everyone who debugs after you — which is why choosing the correct code is a developer responsibility, not a cosmetic detail.

> **Exam:** The four families with fault attribution — 2xx success, 3xx redirection, 4xx **client's** fault, 5xx **server's** fault — plus the anchor codes 200 OK, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error.

> **Pitfall:** Blaming the server for a 404 or the client for a 500. 404 means *the URL asked for something that does not exist* — a client-side addressing mistake. 500 means *the server's own code crashed* — no request, however perfect, could have succeeded.

HTML Tables: Structure, Borders, Padding and Spacing

Day 2 of the lecture moves from concepts to code. Everything from here is hands-on HTML — type it, open it in a browser, and compare with the expected output as you read.

Why tables, and the three-step hierarchy

A table presents data in **rows and columns** — and HTML builds one with a strict three-step nesting hierarchy. This hierarchy is the whole concept; everything else in the chapter is decoration on it:

1. **Outer box:** the `<table>` tag creates the container.
2. **Rows:** `<tr>` (table row) tags go inside `<table>` — one `<tr>` per row.
3. **Columns / cells:** `<td>` (table data) tags go inside each `<tr>` — one `<td>` per cell.

```
<table>                         ← the container (the whole grid)
    <tr>                        ← row 1
        <td> … </td>            ← row 1, cell 1
        <td> … </td>            ← row 1, cell 2
    </tr>
    <tr>                        ← row 2
        <td> … </td>
        <td> … </td>
    </tr>
</table>
```

Read the nesting as containment, because that is what it is: a table *contains* rows; a row *contains* cells; a cell contains content. You never place a `<td>` directly inside `<table>`, and you never place text directly inside `<tr>` — content lives in cells, cells live in rows, rows live in the table.

Worked example: a 3 × 3 table

Target: 3 rows × 3 columns — ID, Name, Phone Number — one heading row and two data rows:

```
<table>
    <tr>
        <td>ID</td>
        <td>Name</td>
        <td>Phone</td>
    </tr>
    <tr>
```

```
        <td>1</td>
        <td>Nitin</td>
        <td>9822011111</td>
    </tr>
    <tr>
        <td>2</td>
        <td>Suresh</td>
        <td>9822022222</td>
    </tr>
</table>
```

What the browser shows (structure correct, styling absent — exactly as raw HTML should look):

ID	Name	Phone
1	Nitin	9822011111
2	Suresh	9822022222

Count to verify: 1 `<table>`, 3 `<tr>`, and 3 `<td>` inside each `<tr>` — 9 cells total. If any row has a different number of cells than the others, the grid goes ragged; HTML tables are built row by row, so *you* are responsible for keeping the per-row cell count consistent.

> **Pitfall: TC is not a valid tag.** The column/cell tag is **TD** — **T**able **D**ata. This sounds trivial until you write `<tc>` in an exam or an interview whiteboard session; the lecture called it out by name because it happens constantly.

Making borders visible: `border`

By default you may see no lines at all, because the **default border is 0 — set by the browser, not by you**. Add the `border` attribute to the `<table>` tag to make borders visible:

```
<table border="1">
```

Two distinct borders appear, and noticing both is the point:

- One border **around the whole table** (the outer box), and
- One border **around each `<td>` cell** (the inner grid).

Raise the number (`border="2"` , …) and the lines thicken. The attribute belongs on `<table>` — never on rows or cells individually at this stage.

Space inside cells: `cellpadding`

`cellpadding` controls the space **inside** each cell — the breathing room between a cell's border and its content:

```
<table border="1" cellpadding="10">
```

Every cell gains inner whitespace on all sides; the text stops hugging the border. The table grows because each cell's interior grew.

Space between cells: `cellspacing`

`cellspacing` controls the space **outside** each cell — the **gap between cells**:

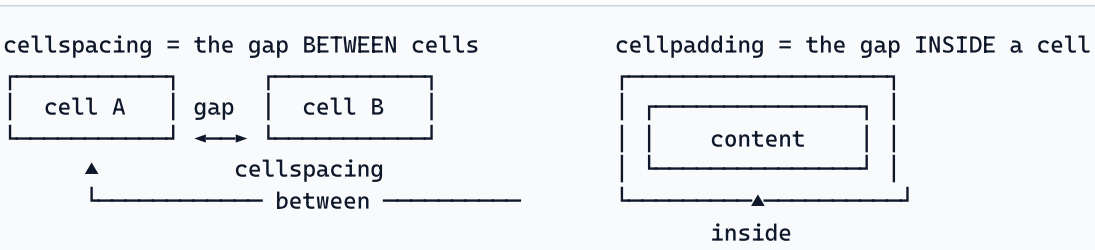
```
<table border="1" cellspacing="10">
```

The cells move apart; the gaps between them widen. And the special value matters: `cellspacing="0"` **removes the default inter-cell gap entirely**, so cell borders sit flush against each other — the clean, modern grid look.

Both attributes are applied **directly on the `<table>` tag**:

```
<table border="1" cellpadding="10" cellspacing="0">
```

Inside vs outside is the entire distinction, so fix it with a picture:



> **Exam:** `cellpadding` = space **inside** the cell (content to cell border); `cellspacing` = space **outside** the cell (cell to cell), with `cellspacing="0"` removing the gap. Both are attributes of `<table>`. Swapping inside/outside is the classic one-mark loss.

Background: `border`, `cellpadding`, and `cellspacing` are old-style **presentational attributes** — styling smuggled into HTML attributes. Modern practice moves all of this into CSS (the subject of the last chapter), but Day 1 teaches the attributes because they make cause-and-effect visible with zero extra machinery, and because you will still meet them in legacy code.

Headings: `<th>` instead of `<td>`

The first row of most tables is not data — it is **headings**. HTML gives header cells their own tag: `<th>` (**table heading**), used in place of `<td>` in the header row:

```
<tr>
    <th>ID</th>
    <th>Name</th>
    <th>Phone</th>
</tr>
```

The browser **automatically renders `<th>` content in bold**, so the header row visually differentiates itself from data rows **without any extra CSS**. Rule of good practice: **always use `<th>` for header cells and `<td>` for data cells** — the browser, screen readers, and future CSS all treat them accordingly.

Semantic structure: `<thead>` and `<tbody>`

Two wrapper tags group the rows by role:

```
<table border="1" cellpadding="10" cellspacing="0">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Phone</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>1</td>
            <td>Nitin</td>
            <td>9822011111</td>
        </tr>
        <tr>
            <td>2</td>
            <td>Suresh</td>
            <td>9822022222</td>
        </tr>
    </tbody>
</table>
```

- `<thead>` wraps the **header row** (the `<tr>` with `<th>` cells).
- `<tbody>` wraps **all the data rows** (the `<tr>`s with `<td>` cells).

There is **no visual change** to the output — and that is precisely the lesson. These are **semantic tags**: same output, better meaning and readability. The benefits are practical, not decorative:

- Code reads better: anyone can see where headings end and data begins.
- CSS or JavaScript can **target the header vs the data sections independently** (style all of `<thead>` one way, `<tbody>` another) instead of chasing individual rows.

Keep this semantic idea in mind — it returns immediately with `<label>` and `<button>`, and it is one of the lecture's recurring themes: **choose the tag that means what the content is, not merely one that looks the same.**

HTML Forms: Purpose, action and method

What a form actually is

A form is how a web page **collects data from the user and submits it to a server** — login, signup, search, checkout, feedback: nearly everything interactive you have ever done on the web went through this mechanism.

But here is the subtle point the lecture builds the chapter on: the `<form>` tag **does not display input fields**. Input fields render perfectly well *without* any `<form>` tag around them. So what does `<form>` add? **Submission behaviour.** The `<form>` tag is the collector and courier: it declares *where* the collected data goes and *how* it travels when the user submits.

Think of it as an envelope. Input fields are the letter's contents — they exist on their own. The `<form>` is the addressed envelope plus the act of posting: without it, the contents sit on the desk forever, visible but never sent.

The two essential attributes

```
<form action="google.com" method="POST">
    <!— input fields live here —>
</form>
```

Attribute	Meaning	Lecture notes
`action`	The **server URL where form data is sent** on submission	`action="google.com"` was the class placeholder; real applications use their **own server's URL**
`method`	The **HTTP request type** — GET, POST, PUT, DELETE	**Default is GET if `method` is not specified**

Both attributes should look familiar: they are exactly the **two mandatory components of every HTTP request** from the HTTP chapter — URL and method. A form submission is not a new mechanism; it is the browser manufacturing an HTTP request out of the user's input. The `action` supplies the URL, the `method` supplies the method, the input fields supply the data, and the submit button (next chapters) triggers the sending.

The default deserves emphasis because it operates everywhere, not just in forms:

> **Exam:** If `method` is not specified, it **defaults to GET**. The **same rule applies to any browser request**: an unspecified method defaults to GET. Typing `https://www.amazon.in` in a browser sends a **GET** request — which is now obvious in hindsight: you are *retrieving* a page, and retrieval is GET's job.

What happens on submit

On a submit-button click, the form **sends a request to the `action` URL using the specified method**, carrying the values from its input fields. Server-side, that request lands exactly like every other request in this course: application logic processes it, the database is consulted or updated, and a response with data and a status code comes back. The form is the frontend end of the client–server story — you have now seen both ends.

The lecture created a practice file, `form-demo.html`, to build this in. The combined worked example near the end of these notes recreates and completes it.

> **Pitfall:** Expecting `<form>` itself to show something on the page. It shows nothing — it is invisible machinery. If your page shows no fields, the missing piece is `<input>` elements (next chapter), not a "broken" form tag.

Input Types, Labels and Line Breaks

One tag, many controls: `<input>` and `type`

The `<input>` tag creates **all standard form controls**; its `type` attribute specifies which kind. One tag to learn, many controls to gain:

`type`	Control	Behaviour and notes
`text`	Single-line plain text field	**Default if `type` is omitted**

`type`	Control	Behaviour and notes
`password`	Password field	**Masks characters** as the user types (encrypted display — dots/asterisks, so shoulders can't surf)
`date`	Calendar date picker	Browser supplies a picker UI; `placeholder` **not supported**
`time`	Time picker	Browser supplies a time UI; `placeholder` **not supported**
`datetime-local`	Date **and** time picker	Both pickers combined in **one control**
`file`	File upload	Shows **"Choose File"** and displays the selected filename; appearance can be **fully restyled with CSS** (e.g. drag-and-drop zones)
`email`	Email field	Like text, but the browser **validates correct email format** (string + `@` + domain) on submit
`number`	Numeric field	**Restricts input to numeric values only**; rejects string input

Patterns to notice rather than memorise row by row: pickers (`date` / `time` / `datetime-local`) trade the hint-text ability for a real selection UI; `email` and `number` bake small validations into the browser for free; and `text` is what you get when you say nothing at all.

Two idea-level notes from the lecture:

- **Browser validation is a convenience, not a wall.** For `email` you may instead use `type="text"` and **validate the format manually in JavaScript** — the alternative exists when you need control the built-in check doesn't give. Either way, recall the earlier principle: client-side checks are for experience; the server re-checks for truth.
- `file` **looks plain only by default.** The humble "Choose File" button can be fully restyled with CSS — the polished drag-and-drop upload zones you see on modern sites are this same control wearing CSS.

The `placeholder` attribute: a hint that disappears

`placeholder` is **hint text shown inside a field before the user types**, which **disappears as soon as input begins**:

```
<input type="text" placeholder="Enter name">
<input type="password" placeholder="Enter password">
```

It answers "what belongs here?" without permanent labels taking space — but it is a *hint*, not a label: it vanishes on typing, and (from the table above) the picker types don't support it at all. Don't use it as the only explanation a field ever gets.

`<label>`: the semantic way to name a field

The `<label>` tag labels input fields:

```
<label>Enter name</label>
<input type="text" placeholder="Enter name">
```

The same *visual* result is achievable with a `<span>` — but `<label>` is the **recommended practice**. Why, if it looks identical? The recurring semantic theme again: a `<label>` **means** "this text names that field", so code is more readable, browsers and assistive technologies understand the association, and future CSS/JS can target labels as labels. Same output, better meaning. Choose the meaningful tag.

`<br>`: putting each field on its own line

By default, consecutive inputs sit side by side on one line. The `<br>` tag is a **line break** — the HTML equivalent of `\n` — used to push each input field onto a new line:

```
<label>Enter name</label>
<input type="text" placeholder="Enter name">
<br>
<label>Enter password</label>
<input type="password" placeholder="Enter password">
```

Two `<br>` tags create **two line breaks**, for extra spacing between fields. It is crude layout by later CSS standards, but it is honest Day-1 machinery: structure first, beauty in the CSS chapter.

> **Exam:** Default `type` when omitted is `text`; `placeholder` is hint text that disappears on typing and is unsupported on `date`/`time`; `<label>` is preferred over `<span>` because it is **semantic** — same look, better meaning.

> **Pitfall:** Using `placeholder` as a permanent label, then wondering why the "label" vanished — it is designed to vanish. And expecting `placeholder` to appear inside a date picker: it won't.

Radio Buttons, Dropdowns, Checkboxes and Textarea

Radio buttons: single choice, enforced by `name`

`<input type="radio">` creates a **circular single-choice button** — the control for "pick exactly one of these", like Male/Female for gender.

The behaviour that surprises everyone the first time:

- **Without a shared `name`, multiple radio buttons can all be selected simultaneously** — each behaves as an independent button, which defeats the entire point of a single-choice control.
- The `name` **attribute, when identical across a group, enforces single selection only.** Selecting one deselects the others in that group.

```
<input type="radio" name="gender"> Male
<input type="radio" name="gender"> Female
```

Both carry `name="gender"`, so **only one can be active**. Give them different `name` values and they become independent buttons with multiple selections possible — *usually wrong* for a radio group.

From first principles: `name` is how the browser knows which buttons belong to the same question. The group is not defined by proximity on the page or by your intention — only by the shared name. (The name matters a second time at submission: it is also the key under which the chosen value travels to the server — preview of why every control in a form wants a meaningful `name`.)

> **Pitfall:** The number-one radio bug — Male *and* Female both selectable. Cause, every time: the `name` attributes differ (or one is missing). Same question ⇒ same `name`.

Dropdowns: `<select>` and `<option>`

A dropdown list takes two cooperating tags:

- `<select>` creates the **dropdown container**.
- Each `<option>` tag defines **one item** in the list.

```
<select>
    <option>10th</option>
    <option>12th</option>
    <option>B.Tech</option>
    <option>M.Tech</option>
</select>
```

Options can be populated **statically** (fixed in the HTML, as above) or **dynamically** — filled by JavaScript based on context, e.g. cascading **country → state → city** dropdowns where each selection determines the next list. The static version is Day 1's; the dynamic behaviour is JavaScript's job, as with all behaviour.

The `value` attribute: display vs submitted data

This is one of the most consequential small ideas in the whole day. On an `<option>`, `value` **separates what is displayed from what is submitted**:

- The **text between the `<option>` tags** is what the user **sees**.
- The **`value="…"` attribute** is what **actually gets sent to the server** on submission.

```
<select>
    <option value="101">10th</option>
    <option value="102">12th</option>
    <option value="103">B.Tech</option>
    <option value="104">M.Tech</option>
</select>
```

The user sees "10th" but the server receives `101`; the user sees "B.Tech" but the server receives `103`. The use case is everywhere in real systems: **the backend expects unique IDs, not display labels.** Display text changes with language, spelling, and redesign; an ID never does. So humans read labels and machines exchange IDs, and `value` is the hinge between the two.

Generalise it, because it applies beyond `<option>`: **what a control shows and what a form submits are two different things, and `value` is how you control the submitted side.** Watch for it again on buttons in the very next chapter.

> **Exam:** "What does the server receive when the user selects 'B.Tech' from `<option value="103">B.Tech</option>`?" — **`103`**, not "B.Tech". If `value` is absent, the display text is what travels — which is exactly why IDs require it.

Checkboxes: the binary control

`<input type="checkbox">` is a **binary check/uncheck control** — something is either accepted or not, with no third state. The canonical use: **"I accept terms and conditions"**, where the checkbox conventionally comes *before* its text, and no `<label>` tag is required before it — the label text can follow directly after the input:

```
<input type="checkbox"> I accept terms and conditions
```

Contrast it with radio deliberately: a checkbox answers one yes/no question and stands alone; radios answer one choose-one question and must be grouped by `name`. Choosing between them is a meaning decision before it is a tag decision.

`<textarea>`: multi-line text

For longer content, single-line `text` runs out. `<textarea>` is a **multi-line text input**:

```
<textarea placeholder="Enter address"></textarea>
```

- Unlike `type="text"`, it **supports line breaks within the same field** (pressing Enter starts a new line instead of submitting).
- **Use cases:** address, comment, review, feedback, message — anywhere the answer is a paragraph, not a word.
- **No character limit by default**; limits are enforced via **JavaScript validation** — e.g. restrict to **255 characters** using JS form validation. HTML provides the field; the rule "at most 255" is behaviour, and behaviour is JavaScript.
- **Supports** `placeholder` (`placeholder="Enter address"`).
- **Automatically gains a scrollbar** as content grows beyond the visible area.
- **Dimensions are adjustable via CSS** — the visible box is a styling decision, separate from how much text it can hold.

Dynamic field visibility

One forward-looking note: making fields **show or hide based on prior input or checkbox state** (check "I have a referral code" → a referral field appears) is handled **entirely in JavaScript, not HTML**. HTML declares the fields that *can* exist; only a behaviour layer can react to the user's choices in real time. Day 1 plants the expectation; the JavaScript sessions harvest it.

Buttons: Plain, Submit and Reset

Three button types

HTML offers **three button types**, differing not in appearance but in how much behaviour is built in:

Type	Behaviour	Default label	Notes
`type="button"` (plain)	**No predefined behaviour** — the developer writes all	Blank button, **no label**	`value` sets the visible label, e.g. `value="Click Me"`

Type	Behaviour	Default label	Notes
	functionality in JavaScript		
`type="submit"`	**Predefined:** sends the form data to the `action` URL using the specified method	**"Submit"** (no `value` needed)	`value` overrides the label — `value="Login"`, `value="Register"`, `value="Sign Up"` — while the functionality stays identical
`type="reset"`	**Predefined:** clears all form fields back to their initial empty state	"Reset"	`value` overrides the label, e.g. `value="Clear Form"`

Unfold the three:

Plain button — a blank slate. It does nothing until JavaScript gives it a job. Even its label is your responsibility: with no `value`, it renders as an empty button. This is the button for custom behaviour — "Calculate", "Preview", "Add another field".

Submit button — the form's trigger. This is the click from the forms chapter: it fires the request to the `action` URL by the form's `method`, carrying the fields' data. Relabelling it (`Login`, `Register`, `Sign Up`) changes only the words on the button — a login form and a registration form are doing the *same mechanical act* of submitting, just to different actions with different fields. Same machinery, different label.

Reset button — the undo. It clears **all** form fields back to their initial empty state. The lecture's walkthrough: after filling gender, date, qualification, and checking terms, clicking reset clears *everything* — one click returns the form to how it first loaded.

The deeper pattern: `submit` and `reset` are behaviours the browser gives you **for free** because they are identical in every form ever built; `button` is the escape hatch for everything else, priced in JavaScript. Knowing which of the three a situation wants is mostly asking "is this behaviour universal (submit/reset) or mine alone (plain)?"

Two ways to create a button

```
<!— Way 1: <input> with the value attribute as the label —>
<input type="submit" value="Login">
<input type="reset" value="Clear Form">
<input type="button" value="Click Me">

<!— Way 2: <button> with the text between the tags (semantic, preferred) —>
<button type="submit">Login</button>
```

```
<button type="reset">Clear Form</button>
<button type="button">Click Me</button>
```

- `<input type="submit/reset/button">` uses the `value` **attribute** for its label — the button is another input control, so it borrows input's labelling mechanism.
- `<button type="submit/reset/button">` puts the **text between the opening and closing tags**. It is the **semantic, preferred** way: `<button>` tags are **more readable and stand out clearly** from the other `<input>` controls in the code. When you scan a form, the buttons announce themselves instead of hiding in a list of inputs.

Same behaviour either way; choose the one a reader thanks you for.

> **Exam:** Three types and their behaviours — **button**: no predefined behaviour (JavaScript supplies it, `value` supplies its label); **submit**: sends form data to `action` by `method`, default label "Submit"; **reset**: restores all fields to their initial state. And the two creation syntaxes, with `<button>` preferred as semantic.

> **Pitfall:** Expecting a plain `type="button"` to submit the form "because it's inside one". It never submits — inside or outside a form, it only does what JavaScript tells it. Submitting is `type="submit"`'s job alone.

CSS Introduction: Selectors, Properties and Where Styles Live

Why CSS exists in this course at all

Until now, every page you have built is structurally correct and visually bare — the HTML-only black-and-white page from the fundamentals chapter. **CSS (Cascading Style Sheets) is used to style and beautify HTML elements**: background colour, font colour, font size, font family, bold, italics, element dimensions, and more. (Historically you have already cheated styling into HTML once today — `border`, `cellpadding`, `cellspacing`. CSS is where that impulse properly lives.)

Recall the division of labour: HTML says what things *are*; CSS says what they *look like*. One document of structure, one set of rules for presentation — change the look without touching the structure, or restructure without rewriting every style.

The syntax: `selector { property: value; }`

All of CSS is one repeated shape:

```
selector { property: value; }
   │           │         │
   │           │         └─ the setting applied (green, 16px, bold, …)
   │           └─ a predefined CSS keyword (color, background-color, font-size, …)
   └─ the HTML element to target — the tag name WITHOUT angle brackets (h1, p, form)
```

- **Selector** — the HTML element to target, written as the **tag name without angle brackets**: `h1`, `p`, `form`. Not `<h1>`, not `<p>`. The selector *refers to* elements; it is not itself an element.
- **Property** — a **predefined CSS keyword** naming the aspect to change: `color`, `background-color`, `font-size`, … You choose from CSS's vocabulary; you do not invent property names.
- **Value** — the setting applied to that property: `green`, `red`, `16px`, …

Multiple properties can be listed inside one selector's curly braces — one rule can restyle several aspects of the same element at once.

Worked example

```css
h1 {
    color: green;
}
```

Read it aloud in the syntax's grammar: "For every `h1` element, set the property `color` to the value `green`." Result: **all `<h1>` text on the page renders green** — one rule, every matching element. That "all of them at once" is the leverage of the selector idea: style is declared per *kind* of element, not repeated per occurrence.

Where CSS is written: `<style>` inside `<head>` — and why

CSS is written inside a **`<style>` tag, placed inside `<head>`**:

```html
<head>
    <style>
        h1 {
            color: green;
        }
    </style>
</head>
<body>
    <h1>Hello World</h1>
</body>
```

The placement is a load-order decision, not a convention to obey blindly. Reason it from first principles:

1. The browser loads `<head>` **before** `<body>`.
2. If the styles are in `<head>`, they are **fully loaded before the body elements render**.
3. So every element appears **already styled** — its first visible moment is its correct one.

The lecture's analogy: **a salon prepares all makeup materials before the client sits down; the client does not wait.** Preparation first, presentation second.

The counterfactual proves the rule: if `<style>` were in `<body>`, styles and elements could load **simultaneously**, causing **unstyled flashes** — the page visibly flickering from bare HTML to styled page as rules arrive. You have seen this flicker on slow websites; now you know its cause. Style in `<head>` eliminates it by construction.

When CSS goes wrong: reading DevTools

CSS fails quietly — a misspelled property doesn't crash anything; the rule is simply ignored. So you need somewhere that shows the failure. That place is the browser's DevTools:

1. **Right-click → Inspect**.
2. Open the **Elements** panel and select the element.
3. The **Styles** section shows the rules applying to it — including **warnings and crossed-out properties**, which are how the browser flags invalid CSS.

Crossed out means "I read this, I could not apply this" — usually a misspelled property or an invalid value. This is the CSS counterpart of the Network tab: the browser is constantly telling you what it did; DevTools is how you listen.

The lecture created `cssdemo.html` for exactly this: an `<h1>Hello World</h1>` styled green via `<style>` in `<head>` — the listing above *is* that file, and the combined example below grows it into a full page.

> **Exam:** CSS syntax is `selector { property: value; }`; the selector is the tag name **without** angle brackets; styles go in a `<style>` tag inside `<head>` **because the head loads before the body**, preventing unstyled flashes.

> **Pitfall:** Writing the selector with angle brackets (`<h1> { color: green; }`) — invalid, silently ignored, and diagnosable only by the crossed-out rule in DevTools. CSS's silence about its own errors makes the Inspect habit non-optional.

Combined Worked Example: The Aligned Form

The exercise

The lecture's assigned exercise: **align the form so every label and input field starts from the same horizontal position**. Anyone who built the Day-2 form field-under-field knows the problem — labels have different lengths ("Enter name" vs "Enter password"), so the input boxes start at ragged, drifting positions.

The hint given in class: **use a `<table>` to structure the form in a rows-and-columns layout** — each label in one `<td>`, its corresponding input in the adjacent `<td>`. Columns align by construction, so every input starts at the same horizontal position: the left edge of column two. The table is being used as a **layout grid**, not to display tabular data — a pragmatic Day-1 technique (CSS will later take over layout entirely, but the alignment *idea* — labels in one column, fields in another — survives every technology change).

The complete solution

One file combining everything from Day 2 of the lecture — the recreated `form-demo.html` and `cssdemo.html` ideas in a single commented listing. Save it, open it in a browser, then break things deliberately (remove a `name`, change a method) and watch DevTools:

```
<!DOCTYPE html>
<html>
<head>
    <title>WPT Day 1 - Registration Form</title>
    <!— CSS lives in <head>: loaded before <body> renders, so no unstyled flash —>
    <style>
        h1 {
            color: green;
        }
        body {
            font-family: Arial, sans-serif;
        }
    </style>
</head>
<body>

    <h1>Student Registration</h1>

    <!— action = where the data goes; method = how it travels (POST: submitting new
data) —>
    <form action="register" method="POST">
        <!— Table = layout grid: labels in column 1, inputs in column 2, all aligned
—>
        <table cellpadding="8" cellspacing="0">
```

```
            <tr>
                <td><label>Enter name</label></td>
                <td><input type="text" name="name" placeholder="Enter name"></td>
            </tr>

            <tr>
                <td><label>Enter password</label></td>
                <td><input type="password" name="password" placeholder="Enter
password"></td>
            </tr>

            <tr>
                <td><label>Enter email</label></td>
                <!-- type="email" validates string + @ + domain on submit -->
                <td><input type="email" name="email" placeholder="Enter email"></td>
            </tr>

            <tr>
                <td><label>Enter age</label></td>
                <!-- type="number" rejects non-numeric input -->
                <td><input type="number" name="age" placeholder="Enter age"></td>
            </tr>

            <tr>
                <td><label>Date of birth</label></td>
                <!-- date picker: placeholder is NOT supported here -->
                <td><input type="date" name="dob"></td>
            </tr>

            <tr>
                <td><label>Upload photo</label></td>
                <td><input type="file" name="photo"></td>
            </tr>

            <tr>
                <td><label>Gender</label></td>
                <td>
                    <!-- same name="gender" on both = only ONE can be selected -->
                    <input type="radio" name="gender" value="M"> Male
                    <input type="radio" name="gender" value="F"> Female
                </td>
            </tr>

            <tr>
                <td><label>Qualification</label></td>
                <td>
                    <!-- user sees the text; the server receives the value (101, 102,
... ) -->
                    <select name="qualification">
                        <option value="101">10th</option>
```

```
                        <option value="102">12th</option>
                        <option value="103">B.Tech</option>
                        <option value="104">M.Tech</option>
                    </select>
                </td>
            </tr>

            <tr>
                <td><label>Address</label></td>
                <!-- textarea: multi-line, placeholder supported, scrollbar appears as
it grows -->
                <td><textarea name="address" placeholder="Enter address"></textarea>
</td>
            </tr>

            <tr>
                <td></td>
                <td><input type="checkbox" name="terms"> I accept terms and
conditions</td>
            </tr>

            <tr>
                <td></td>
                <td>
                    <!-- submit sends the form to action by method; reset restores
empty fields -->
                    <button type="submit">Register</button>
                    <button type="reset">Clear Form</button>
                </td>
            </tr>

        </table>
    </form>

</body>
</html>
```

Trace it end to end

Reading the solution against the whole day's concepts — this is the revision pass disguised as a walkthrough:

1. **Browser as client.** Opening this file renders the form; nothing has touched a server yet. The moment Register is clicked, the browser becomes the client of Chapter 2 and manufactures a request.
2. **The request's two components.** `action="register"` supplies the URL, `method="POST"` supplies the method — POST because, per the methods table, registration is *submitting new data*.

3. **Choice controls behave as designed.** Gender allows exactly one selection (shared `name="gender"`), and the server receives `M` / `F` and `103`-style qualification codes — the `value` attributes, not the display words.
4. **Browser-side validation fires first.** A malformed email or text in the age field is caught by the `email` / `number` types in the browser before anything travels — JavaScript's and the server's checks remain as later lines of defence.
5. **The response closes the loop.** The server processes, stores to the database, and replies with data plus a status — `200 OK` if all went well, a `400` if the data was wrong, a `500` if the server's code broke. The browser displays whatever came back.
6. **Reset demonstrates built-in behaviour.** Clear Form empties every field — no JavaScript written, none needed, because `type="reset"` carries its behaviour with it.
7. **CSS did its work invisibly.** The green heading appeared styled on first paint because the `<style>` block sat in `<head>`, loaded before the body rendered — no unstyled flash.

And the small standalone echoes of the lecture files, for completeness — `cssdemo.html` reduced to its essence is just steps already inside the example: an `<h1>Hello World</h1>` plus the `h1 { color: green; }` rule in `<head>`. If you can explain why each piece of the big file sits where it sits, Day 1 is yours.

Quick Revision Checklist

Topic	One-line takeaway
Server	Always-on, publicly reachable computer hosting the application **and** its database; same architecture as a laptop, far more capacity
Why a server	**Accessibility** (world can't use your laptop) + **scale** (lakhs of TBs, high-throughput processing) — both required
Client	The **browser**, not the end user; the user interacts with the browser, the browser with the server
Request flow	User → browser → request → server logic → DB query → data → response → browser renders
No client, no work	The server only reacts to requests — a restaurant with zero customers cooks nothing
Frontend	Everything the user sees/interacts with in the browser (navbar, buttons, forms, images)

Topic	One-line takeaway
Backend	**Application logic + database**; full app = UI + logic + database
HTML	Structure/skeleton; without it nothing appears; alone it's plain black-and-white
CSS	All styling — "a beauty parlour for your web page"
JavaScript	Client-side logic/interactivity in the browser; makes button clicks *do* things; validation, dynamic content, events
Java vs JavaScript	Completely unrelated — "car vs carpet"
Libraries/frameworks	Built **on top of** HTML/CSS/JS (React, Angular, Vue, jQuery, Tailwind, Bootstrap); fundamentals stay non-negotiable
Backend languages	Java, Python, .NET, PHP, C/C++, and now JavaScript (course: JavaScript for both layers)
Databases	Relational (MySQL, PostgreSQL, Oracle) vs non-relational (MongoDB, Cassandra); **no forced pairing** with language; architect decides
Every request	**URL/IP (where)** + **method (what for)** — both always present
GET / POST / PUT / DELETE	Retrieve / submit new / update existing / remove
Method default	Unspecified method ⇒ **GET** (forms and typed URLs alike)
Every response	Data (**optional**) + status (**always**); status = code + message, predefined and universal
Status families	2xx success · 3xx redirection · 4xx **client's** fault · 5xx **server's** fault
Anchor codes	200 OK · 400 Bad Request · 401 Unauthorized · 404 Not Found · 500 Internal Server Error
Table hierarchy	`<table>` → `<tr>` → `<td>`; **TC is not a tag**
Table spacing	`border` on `<table>`; `cellpadding` = space **inside** cells; `cellspacing` = space **between** cells (`0` removes the gap)
Header cells	`<th>` (bold automatically) for headers, `<td>` for data; `<thead>` / `<tbody>` are semantic wrappers
`<form>`	Displays nothing itself; adds submission — `action` (URL) + `method` (default GET)

Topic	One-line takeaway
Input defaults	`type` omitted ⇒ `text`; `placeholder` = disappearing hint (unsupported on date/time)
Validation types	`email` checks string + `@` + domain; `number` rejects non-numeric input
Radio rule	Same question ⇒ same `name`, or multiple selections become possible
`value` on `<option>`	User sees the text; the **server receives the** `value` (10th → `101`)
Textarea	Multi-line; no default character limit (JS enforces, e.g. 255); scrollbar automatic; size via CSS
Buttons	`button` = no built-in behaviour · `submit` = sends form (default label "Submit") · `reset` = clears all fields
Button syntax	`<input … value="label">` or semantic `<button>label</button>` (preferred)
CSS syntax	`selector { property: value; }` — selector has **no angle brackets**
Where CSS lives	`<style>` inside `<head>` — head loads first, so no unstyled flash; check crossed-out rules in DevTools → Styles

---

Practice Exercises

Work these without looking at the answers. They are ordered by chapter, and most are deliberately about the exact confusions the lecture called out.

Concepts

1. A friend says: "My laptop is powerful, so I'll just host our class project on it and give everyone my IP." Using the two reasons a server is necessary, list everything wrong with this plan.
2. In the sentence "the user sent a request to Flipkart", identify the technical error and rewrite the sentence correctly.
3. A page shows a button that does nothing when clicked. The button's HTML and CSS are both perfect. Which of the three frontend technologies is missing, and what is its job?
4. True or false, with a one-line justification: "Our backend is JavaScript, so our database has to be MongoDB."
5. Who chooses the technology stack in a real company, and on the basis of what?

HTTP

6. For each operation, name the correct HTTP method: (a) loading the homepage, (b) creating a new account, (c) changing your saved address, (d) deleting a comment, (e) searching for "Samsung S25".
7. A request fails with status `404`. Whose fault is it, what does it mean, and what do you check first?
8. During testing your login form returns `500`. During the same test, a deliberately empty submission returns `400`. Explain what each code tells you about where the problem lives.
9. A form is written as `<form action="save">` with no `method`. Which method is used when it is submitted, and why do you know?
10. "The response contained no data, so it must have failed." Correct the misunderstanding in one or two sentences.

HTML tables and forms

11. Write the HTML for a 2-row × 2-column table (headers: Roll, Name; one data row: 1, Asha) using `<th>`, `<thead>`, and `<tbody>`, with visible borders and no gap between cells.
12. Spot the bug: two radio buttons for "Yes" and "No" can both be selected at once. The code is `<input type="radio" name="yes"> Yes <input type="radio" name="no"> No`. What is wrong and what is the fix?
13. A user selects "M.Tech" from `<option value="104">M.Tech</option>`. What exactly does the server receive? What would it receive if the `value` attribute were removed?
14. Predict the behaviour: a user fills an entire long form, then clicks a button written as `<input type="button" value="Submit Form">`. What happens, and why? What one-word change fixes it?
15. You need (a) a field for a 200-word feedback message, and (b) a field where typed characters must be hidden. Name the correct control for each.
16. In `h1 { colour: green; }` nothing turns green and nothing crashes. Where in the browser do you look to confirm what went wrong, and what will you see?

Practice Answers

1. **Both server reasons are violated.** *Accessibility:* the laptop is behind a home router with a changing address, and it sleeps/shuts — the world cannot reliably reach it. *Scale:* one consumer machine cannot store/process the combined data and simultaneous load of all users. (Electricity/internet outages break the 24/7 condition too.)

2. The **browser** is the client that sends requests; the user only interacts with the browser. Correct: "The user's browser sent a request to the Flipkart server, and displayed the response to the user."
3. **JavaScript.** HTML displays the button and CSS styles it, but only JavaScript supplies behaviour — something happening on click. (Or, if the intent was form submission, the button should be `type="submit"` — built-in behaviour, no JS needed. Either diagnosis earns the mark; "rewrite the CSS" does not.)
4. **False.** There is no forced pairing between backend language and database; a JavaScript backend can use MySQL, and the choice is driven by application requirements.
5. The **software architect**, evaluating the project and customer requirements, selects the frontend framework, backend language, and database combination.
6. (a) **GET** — retrieving a page. (b) **POST** — submitting new data. (c) **PUT** — updating existing data. (d) **DELETE** — removing data. (e) **GET** — fetching/searching for data.
7. **The client's fault** (4xx family). `404 Not Found` means the requested resource does not exist at the given URL — check the URL/address in the request first.
8. `500` says the **server's code crashed** on a legitimate submission — debug the server. `400` says the **client sent malformed/incorrect data** — the empty submission was rightly rejected. Same form, opposite sides at fault.
9. **GET.** An unspecified method defaults to GET — in forms and in browser requests generally.
10. A response always contains a **status**; only the **data** is optional. A response with no data can still be a complete success or an informative failure — judge it by the status code, not by the presence of data.

11. Here is the required table (`cellspacing="0"` is the "no gap between cells" requirement; `border="1"` makes borders visible):

```
<table border="1" cellspacing="0">
    <thead>
        <tr>
            <th>Roll</th>
            <th>Name</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>1</td>
            <td>Asha</td>
        </tr>
    </tbody>
</table>
```

12. The two buttons have **different** `name`**s** (`yes` vs `no`), so the browser treats them as independent buttons. Fix: give both the **same** name, e.g. `name="answer"` — a shared `name` is what enforces single selection.

13. The server receives `104`. Without the `value` attribute, it would receive the display text, `M.Tech`.

14. Nothing happens. `type="button"` has no predefined behaviour and no JavaScript was attached. Change `button` to `submit` so the click sends the form data to the `action` URL by the form's method.

15. (a) `<textarea>` — multi-line input for long content. (b) `<input type="password">` — masks characters as they are typed.

16. DevTools → right-click → Inspect → Elements → Styles. The invalid property appears **crossed out** with a warning: CSS fails silently, and the Styles panel is where the browser reports it. (`colour` is not a CSS property — the keyword is `color`.)

> Coverage: WPT Day 1 (21–22 Oct 2026) — what a server is and why it is necessary, client–server architecture, frontend vs backend, HTML/CSS/JavaScript, libraries and frameworks, backend and database choices, HTTP request methods, HTTP status codes, HTML tables, HTML forms and input types, radio/dropdown/checkbox/textarea controls, button types, and the first CSS rules — every concept rebuilt from first principles, with the lecture's own examples, analogies, and live demonstrations preserved.